MicroBlaze V ソフト プロセッサの実装

2025.2

概要

この演習では、AMD Spartan™ UltraScale+™ FPGA SCU35 評価キットをターゲットとした AMD MicroBlaze™ V ソフト プロセッサ デザインの実装方法を説明します。

本演習では、AMD Vivado™ Design Suite で MicroBlaze V Preset サンプル プロジェクトを開き、設定し、デバイス イメージを生成する方法、およびハードウェアをエクスポートする方法について説明します。

また、AMD Vitis™ Unified IDE テンプレートを使用してシンプルなアプリケーションを作成し、SCU35 ボード上で実行する方法についても説明します。

この演習の所要時間は約 60 分です。

CloudShare ユーザーのみ

ラボを完了するために 3 回の試行が提供されます。各ラボの完了に割り当てられた時間は、想定所要時間の 2 倍です。タイマーは一度開始すると、一時停止することはできません。ラボを試行するたびに前回の試行がリセットされ、前回の作業内容は保存されません。

一部の演習はハードウェアベースであり、演習の一部またはすべてを実行するためにボードが必要です。通常、ハードウェアを必要とする演習では、最後の手順でのみターゲット ボードが必要となります。ターゲット ボードをクラウドベースの環境に直接接続する方法がないため、CloudShare 環境ではハードウェアベースの演習のこれらの部分に対応していません。したがって、これらの手順はローカルで実行する必要があります。ハードウェアベースの演習のこれらのセクションを完了するには、指定された評価ボードと、ご使用のマシンにローカルでインストールされた AMD ツールが必要です。評価ボードをお持ちでない場合は、その特定の手順や演習の内容を確認するのみとなります。

詳細については、On-Demand Portal の FAQ にある [On-Demand Labs] セクションの「Can I run the labs on my local machine?」の質問を参照してください:
https://www.amd.com/en/training/customer/adaptive-computing/faq.html

© Copyright 2026 Advanced Micro Devices, Inc.

目的

このラボを完了すると、次のことができるようになります:

- Vivado Design Suite で MicroBlaze V Preset サンプル プロジェクトを開く
- MicroBlaze V IP コアで利用可能なさまざまなコンフィギュレーション オプションを特定します
- デバイス イメージを生成し、ハードウェア プラットフォームをエクスポート
- Vitis Unified IDE で提供されているサンプル テンプレートを使用して、ベアメタル アプリケーションを作成します。
- Spartan UltraScale+ SCU35 ボードでアプリケーションを実行する

はじめに

MicroBlaze V プロセッサは、AMD アダプティブ SoC および FPGA への実装向けに設計された 32 ビット RISC-V ソフト プロセッサ コアです。このプロセッサは、独自の MicroBlaze アーキテクチャからオープンな RISC-V 命令セット アーキテクチャ (ISA) への進化を遂げたものであり、ライセンスの制約なしに、オープン スタンダードに基づく組み込みプロセッシング ソリューションを開発者に提供します。

完全にプログラマブル ロジック ファブリック内に実装される MicroBlaze V プロセッサは、乗除算 (M)、アトミック操作 (A)、圧縮命令 (C)、ビット操作 (B) などのコンフィギュレーション可能な RISC-V 拡張機能によって高い柔軟性を提供します。このコンフィギュレーション機能により、設計者は最小限のリソース面積、確定的なリアルタイム性能、計算スループットなど、特定の要件の優先順位に合わせて最適化できます。

このプロセッサは、Vivado Design Suite および Vitis Unified IDE を通じて AMD 開発エコシステムと統合されており、使い慣れたツールを使用した開発が可能です。

主な機能は次のとおりです。

- RISC-V オープンソース ISA に準拠: 業界全体の包括的なソフトウェアおよびソリューションのエコシステムによって支えられており、既存の MicroBlaze プロセッサ デザインからの容易なハードウェア移行と、RISC-V デザインのソフトウェア ポータビリティを実現します。
- 構成可能な ISA サポート: RV32I および RV64I 基本整数命令セット、ならびにオプションの M、A、F、C、ビット操作 (Zba、Zbb、Zbc、Zbs) 拡張機能をサポートし、コード圧縮によってコード サイズを大幅に縮小してデザイン メモリを節約できます

© Copyright 2026 Advanced Micro Devices, Inc.

- 効率的なアーキテクチャ: マイクロコントローラーからアプリケーション プロセッサに至るさまざまなアプリケーションに対応する選択可能な複数のプリセット コンフィギュレーションを備え、面積やパフォーマンスを最適化する 4 つのパイプライン オプションを提供し、セーフティクリティカル システム向けにデュアルコア ロックステップや TMR などのオプションの安全対策を統合
- 高度な設計柔軟性: Vivado および Vitis ツールに完全に統合された設計フローにより、すべての AMD アダプティブ SoC および FPGA に対応し、使いやすい GUI または CLI インターフェイスを提供するとともに、統合 MicroBlaze V プロセッサ サブシステム向けに最適化された IP によってシステム コンポーネント数を削減します。

このラボでは、Vivado のサンプル プロジェクト フローを使用して、Spartan UltraScale+ FPGA SCU35 評価キット上に MicroBlaze V ソフト プロセッサを実装します。さまざまなプロセッサ コンフィギュレーション オプションを確認し、ハードウェア プラットフォームを生成して、Vitis Unified IDE を使用して "Hello World" ベアメタル アプリケーションを開発します。

このハンズオン演習は、AMD FPGA 内で RISC-V ベースのソフト プロセッサの柔軟性を活用する組込みシステムを開発するための基礎を築きます。

ラボ環境の理解

本コースで提供されるラボおよびデモは、Linux® プラットフォーム上で動作するように設計されています。

1 つの環境変数 TRAINING_PATH が必要です。これは演習ファイルの場所を示します。この変数は CloudShare/CustEd_VM 環境にあらかじめ設定されています。

一部のツールはこの環境変数を直接使用できますが (つまり、$TRAINING_PATH が認識されて自動的に展開される)、手動での展開が必要なツールもあります (CloudShare/CustEd_VM 環境の場合は /home/amd/training)。各ツールでの対応手順については、ラボの手順書に記載されています。その他の環境では、スクリプトが正常に動作するためにこの変数を定義する必要があります。

Vivado™ Design Suite および Vitis™ Unified IDE は、多くのラボで使用される Tcl 環境を提供します。いずれかのツールを起動すると、以前にツールを起動したときの proc や変数が一切残っていない、クリーンな Tcl 環境で起動します。

Tcl スクリプトを source した場合、または Tcl 変数を手動で設定した後にツールを閉じた場合、ツールを再度開いた際には、Tcl スクリプトを再度 source し、演習に必要な変数を再設

© Copyright 2026 Advanced Micro Devices, Inc.

定する必要があります。これはターミナル ウィンドウでも同様であり、新しいターミナルを開くとすべての変数設定がクリアされます。

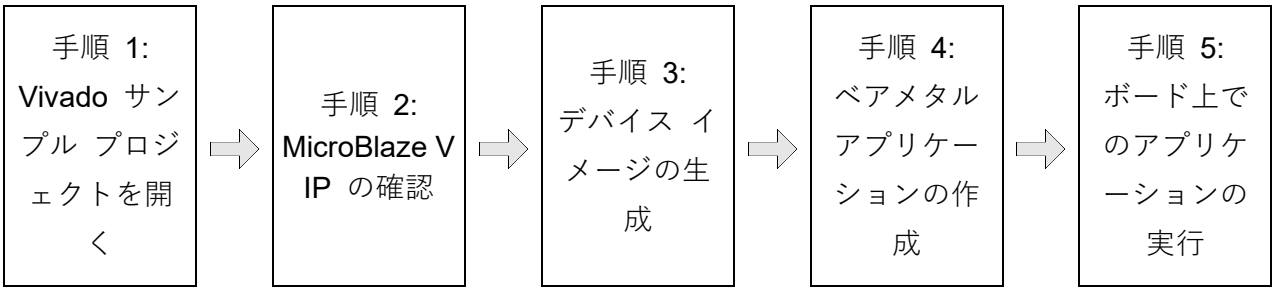
命名法

各種引数の使用方法を説明するために、正式な表記規則が使用されています。以下に、よく使用される記号の一部を示します。

シンボル	説明	例	説明
<text>	フィールドを示します	`cd <dir>`	`<dir>` はディレクトリ名を表します。`<` および `>` 記号は入力しません。移動先のディレクトリが `XYZ` の場合は、環境に `cd XYZ` と入力します。
[text]	オプションの引数を示します	`ls [ | more]`	これは ls <Enter> または ls | more <Enter> と解釈できます。最初の例は現在の Linux ディレクトリ内のファイルを一覧表示し、2 番目の例は現在の Linux ディレクトリ内のファイルを一覧表示しますが、さらに対象の出力を more ツールに渡してページ分割して表示します。ここで、パイプ記号 (|) は Linux の演算子です。
|	選択肢を示します	`cmd <ZCU104 | VCK190>`	cmd コマンドは 1 つの引数を取り、ZCU104 または VCK190 のいずれかを指定できます。cmd ZCU104 または cmd VCK190 のいずれかを入力します。

© Copyright 2026 Advanced Micro Devices, Inc.

全体の流れ



Vivado Design Suite のサンプル プロジェクトを開く 手順 1

この手順では、まず Vivado Design Suite を起動し、[Getting Started] ページの [Open Example Project] リンクからサンプル プロジェクトを開きます。

Vivado Design Suite を開くには、次の 2 つの方法があります。

1-1. [Vivado Design Suite] を開きます。

1-1-1. **[taskbar]** から **[Vivado]** アイコン **()** をクリックします。

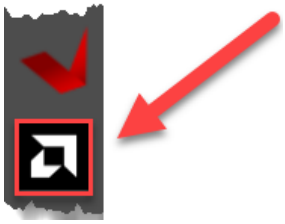


図 1-1: Taskbar からの Vivado Design Suite の起動

注記: 起動には少し時間がかかります。お使いの環境によってはアイコンの並び順が異なる場合があります。

または、[Linux terminal] ウィンドウ (<Ctrl + Alt + T>) を開き、次のように入力します:

```
source /opt/amd/2025.2/Vivado/settings64.sh; vivado
```

注: このインストール パスは、CustEd VM および CloudShare 環境で有効です。お使いの環境に適したパスを使用してください。

© Copyright 2026 Advanced Micro Devices, Inc.

ツールを起動すると、[Welcome] ウィンドウが表示されます。ここから、新規プロジェクトの作成、既存プロジェクトのオープン、Tcl コマンドの入力、ドキュメントやサンプルへのアクセスが可能です。

1-1-2. **[オプション]** 表示される情報が多いため、ウィンドウを最大化します。

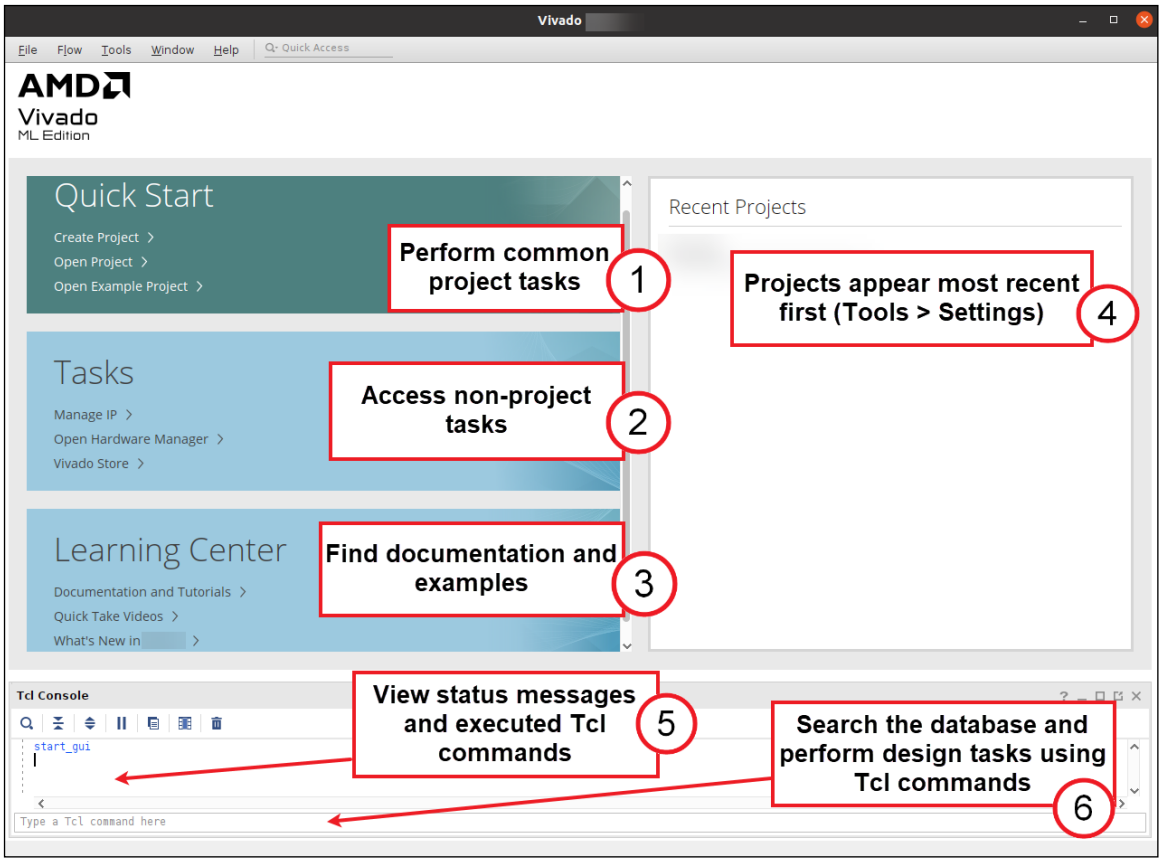


図 1-2: Vivado Design Suite Welcome Screen

ヒント: [Tcl Console] が表示されていない場合は、[Tcl Console] タブをダブルクリックして表示してください。

© Copyright 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

[New Project Wizard] は、プロジェクト ファイル (.xpr) を作成します。このプロジェクト ファイルは、デザイン ファイルを整理し、デザイン入力からデバイス プログラミングおよびデバッグに至る開発プロセスの各フェーズのステータスを追跡します。

[Open Example Project] は、AMD が厳選したテンプレートから開始するための最も迅速な方法です。あらかじめ構成された Vivado IP integrator デザインを含む、すぐに使用可能な Vivado プロジェクトを作成し、選択したボード プリセットおよびデフォルトの制約を適用するため、手動でセットアップを行うことなく直接ビルドおよびエクスポートに進むことができます。

1-2. Vivado Design Suite の新規サンプル プロジェクトを作成します。

1-2-1. **[Open Example Project]** をクリックしてプロセスを開始します **(1)**。

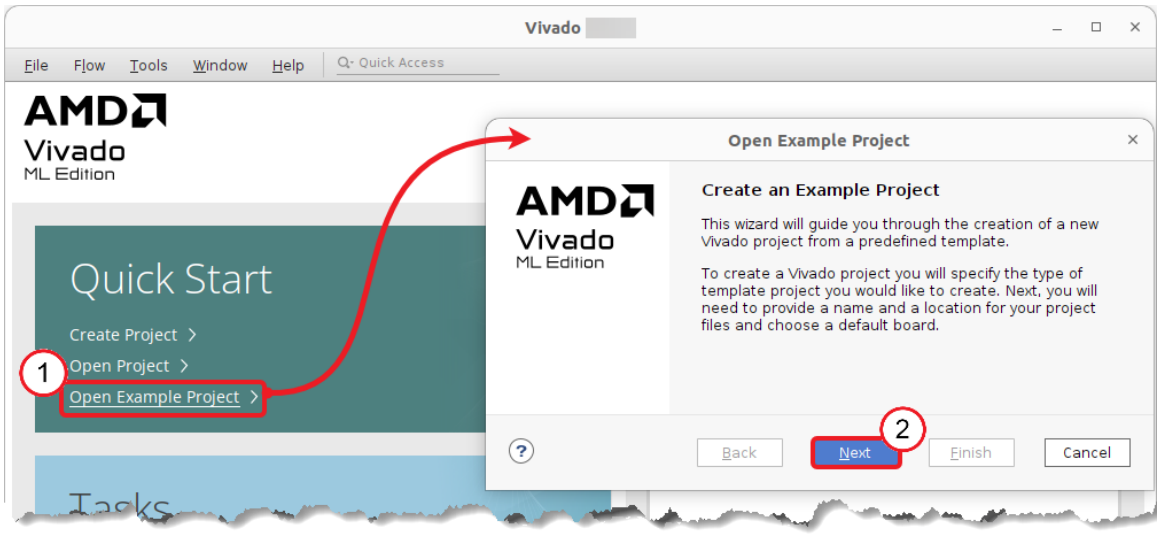


図 1-3: Vivado Design Suite サンプル プロジェクトを開く

これにより、[Open Example Project Wizard] が起動します。

1-2-2. **[Next]** をクリックして導入ダイアログ ボックスを終了し、プロジェクト固有の情報の入力を開始します **(2)**。

© Copyright 2026 Advanced Micro Devices, Inc.

1-3. プロジェクト テンプレートを選択します。

1-3-1. [MicroBlaze V Design Presets] テンプレートを選択します (1)。

[Description] ボックスに MicroBlaze V デザインに関する詳細が表示されます。

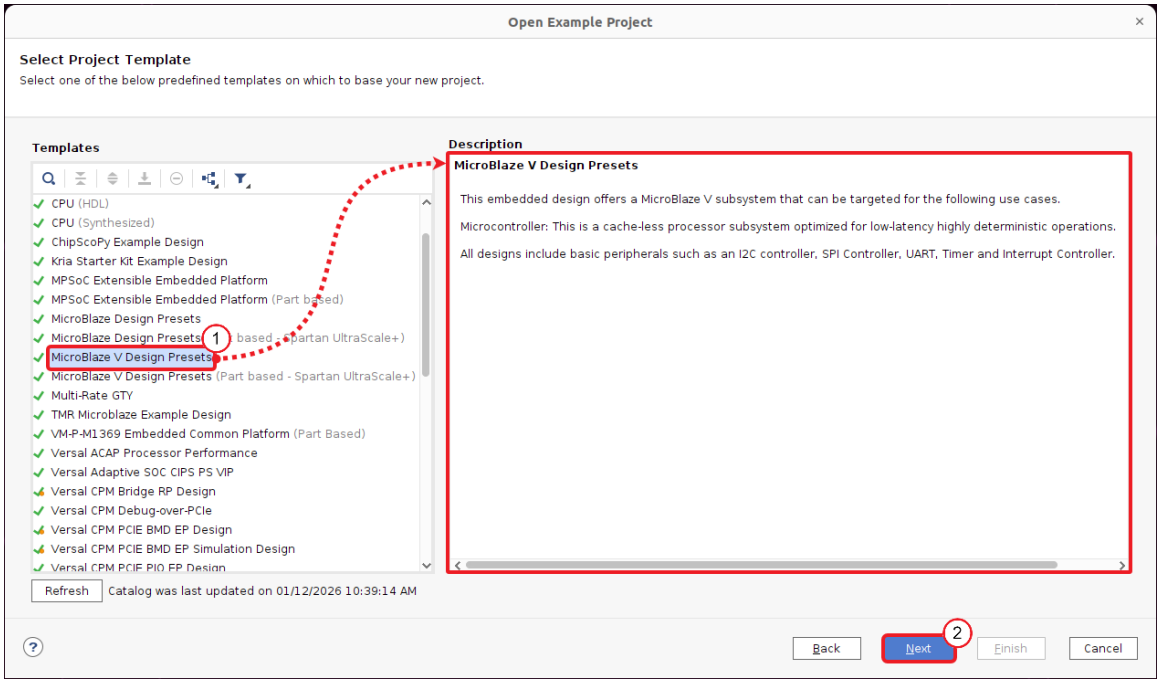


図 1-4: MicroBlaze V Design Template の選択

1-3-2. [Next] をクリックして、プロジェクト固有情報の入力を開始します (2)。

© Copyright 2026 Advanced Micro Devices, Inc.
AMD together we advance_

1-4. プロジェクトのさまざまな側面について説明します。

1-4-1. **[Project name]** フィールド **(1)** に **mbv_hardware** と入力します。

1-4-2. **[Project location]** フィールド **(2)** に次の場所を入力します**:**

```
$TRAINING_PATH/microblaze_v/lab/hw
```

重要: Vivado Design Suite は、Linux および Windows 環境の両方での実行時に変数を展開できます。使用可能な環境変数には (OS に関係なく) 先頭に '$' 記号を付ける必要があります。

または、[Browse] 機能を使用して、プロジェクトを配置する場所に移動することもできます。

1-4-3. **[Create Project Subdirectory]** オプションが選択されている場合は、選択を解除します **(3)**。

これをチェックしたままにすると、演習で不要な階層が作成されます。

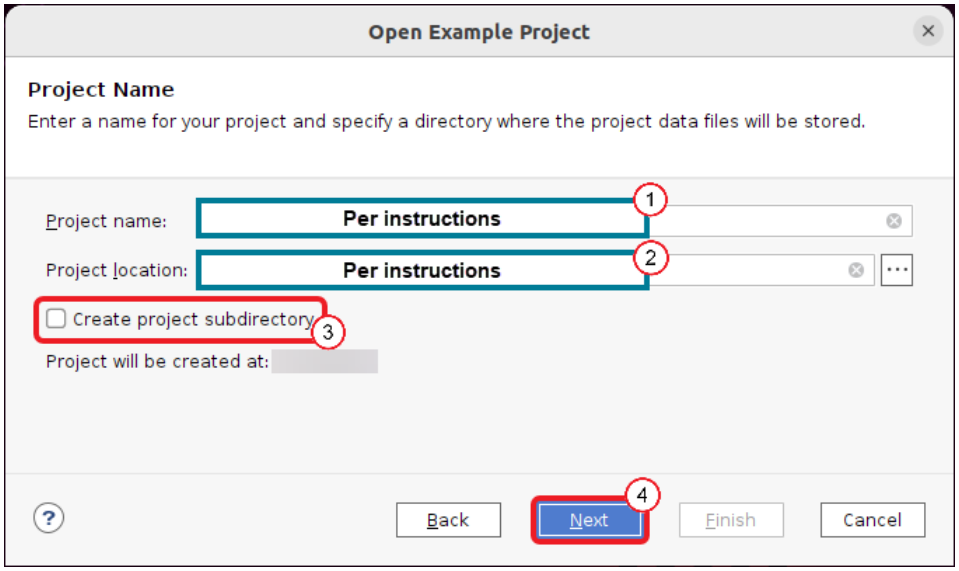


図 1-5: Project Name および Location の入力

1-4-4. **[Next]** をクリックして、ターゲット デバイス**/**プラットフォーム選択 **(4)** に進みます。

© Copyright 2026 Advanced Micro Devices, Inc.

1-5. ボード名でフィルタリングして、ターゲット パーツを選択します。

1-5-1. **[Search]** オプションを使用して、**Spartan UltraScale+ SCU35 Evaluation Platform** を検索します **(1)**。

1-5-2. リストから **[Spartan UltraScale+ SCU35 Evaluation Platform]** ボードを選択します。

誤ってハイパーリンクをクリックしてしまった場合、そのボードの Web ページが開きます。ブラウザー ページは閉じて問題ありません。

使用するボードがすぐに表示されない場合は、[Refresh] ボタンをクリックしてボードカタログを更新してください。

注記: この Web ページにはボードに関する重要な情報やリソースが含まれていますが、この演習を完了するためにこれらの詳細は必要ありません。

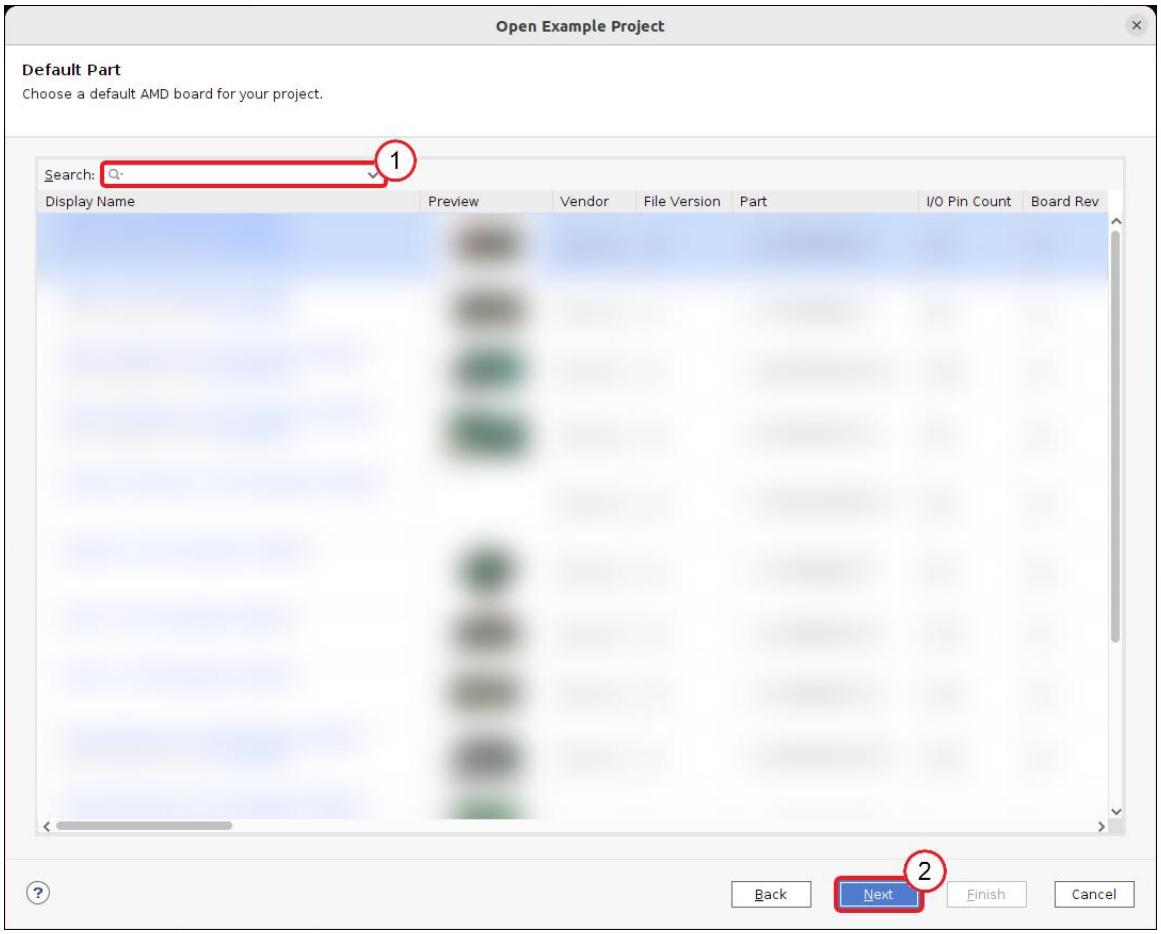


図 1-6: プロジェクトのボードの選択

1-5-3. **[Next]** をクリックして、デザイン プリセット **(2)** を選択します。

© Copyright 2026 Advanced Micro Devices, Inc.
AMD together we advance_

1-6. デザイン プリセットを選択します。

1-6-1. **[Microblaze V Preset Configurations] (1)** の下にある **[Microcontroller]** を選択します。

注: ベアメタル アプリケーションを実行するハードウェアを作成するためにこれが選択されています。RTOS アプリケーションを実行するには、[Real-time] オプションを選択できます。

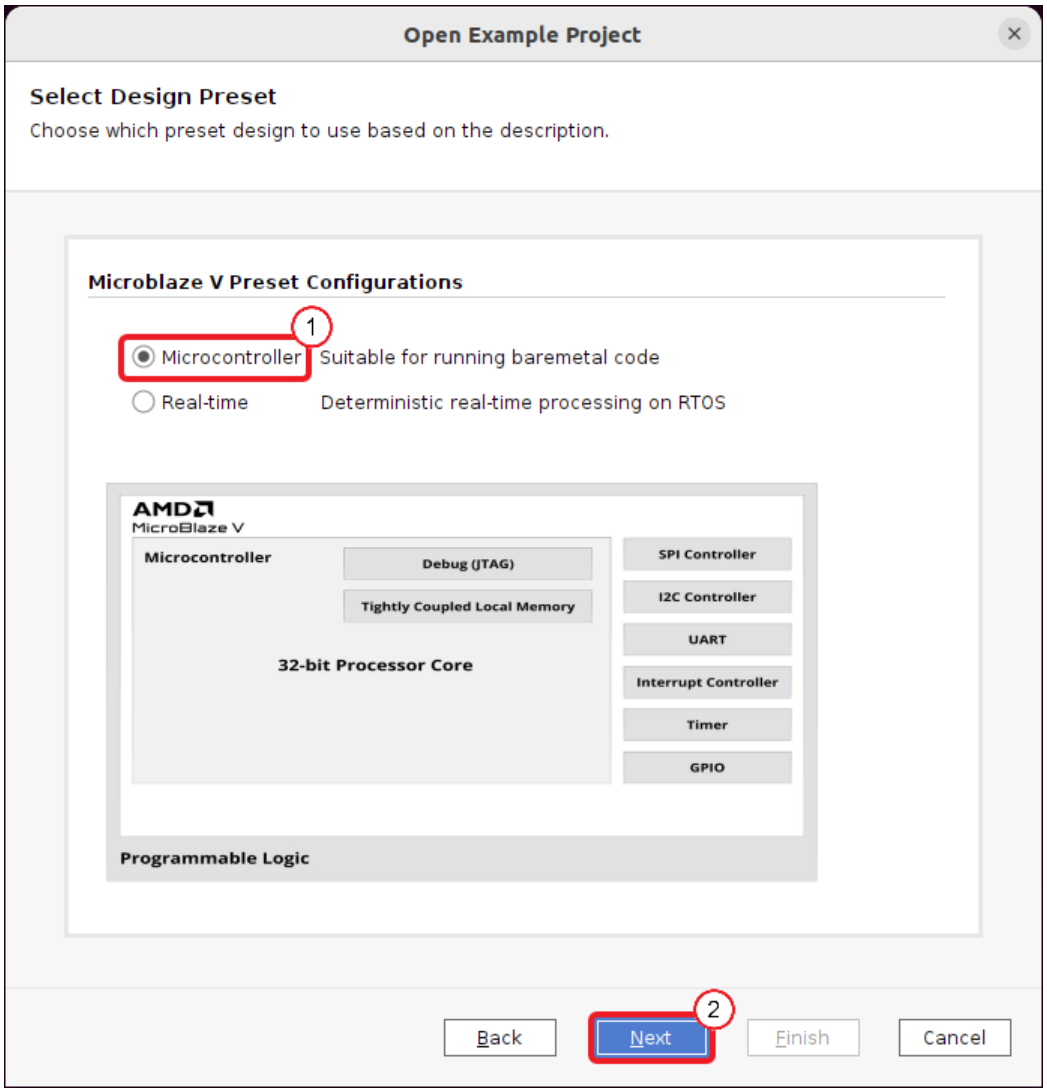


図 1-7: Microcontroller Design Preset の選択

1-6-2. **[Next]** をクリックして、サマリ **(2)** に進みます。

プロジェクトのサマリが表示されます。入力した情報を変更する場合は、該当するダイアログ ボックスに戻るまで [Back] をクリックします。プロジェクトの作成後も、プロジェクト プロパティは編集可能です。

1-6-3. **[Finish]** をクリックしてこれらの設定を適用し、プロジェクトをビルドします。

© Copyright 2026 Advanced Micro Devices, Inc.

プロジェクトが作成され、Vivado Design Suite GUI の操作画面が表示されます。

MicroBlaze V IP の確認 手順 2

この手順では、MicroBlaze V IP のカスタマイズ オプションを確認します。これらのオプションはプロセッサ アーキテクチャやシステム統合の動作を定義するため、これらを理解しておくことは今後の設計において重要です。

2-1. [MicroBlaze V] IP を開きます。

2-1-1. ブロック ダイアグラムがまだ開いていない場合は、**[Flow Navigator]** の **[IP Integrator]** にある **[Open Block Design]** オプションをクリックします。

ブロック ダイアグラムは、次の図のようになります。

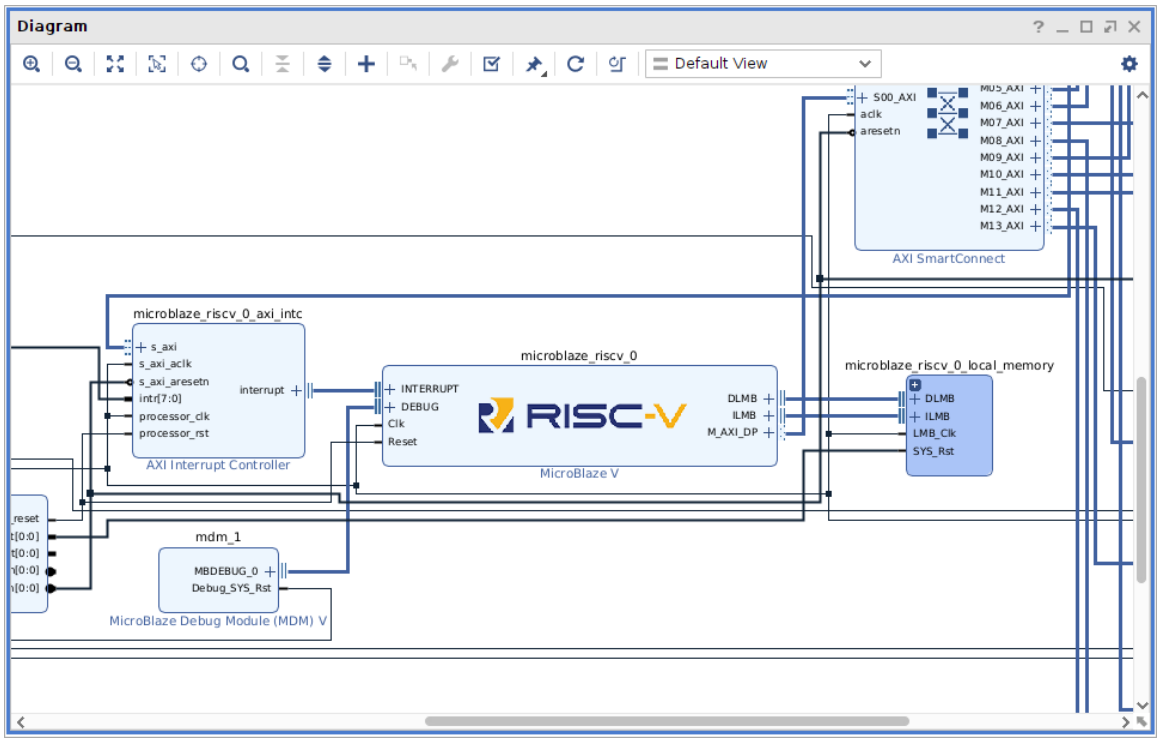


図 1-8: MicroBlaze V IP コア

2-1-2. **[Diagram]** ウィンドウで **[MicroBlaze V]** プロセッサ ブロックをダブルクリックします。

これにより、[MicroBlaze V IP Configuration Wizard] の [Welcome] ページが開きます。

© Copyright 2026 Advanced Micro Devices, Inc.
AMD together we advance_

MicroBlaze V IP は非常に柔軟性が高く、さまざまな方法でコンフィギュレーションできるため、多くの動作パラメーターを設定できます。

[Welcome] ページでは、高レベルのハードウェア/ソフトウェア コントラクトを処理し、次の設定が含まれています。

- **[Predefined Configurations]: さまざまな事前定義されたコンフィギュレーションから選択します。選択内容に基づいて、ウィザード全体の特定のオプションが設定されます。デフォルトでは、[Microcontroller Preset] コンフィギュレーションが選択されています (1)。利用可能なプリセットは次のとおりです。**
  - **[Microcontroller Preset]: RV32IMC ベースのマイクロコントローラー設計に適しています**
  - **[Real-Time Preset]: リアルタイム制御向けであり、ユーザー モードおよび物理メモリ保護を備えた RV32IMAC をベースにしています。**
  - **[Application Preset]: 高性能アプリケーション向けに設計されており、スーパーバイザー モードおよびページベースの仮想メモリを備えた RV32IMAFC をベースとしています。**
  - **RV32IMC: 整数の乗算および除算、ならびに圧縮の標準拡張を備えた RISC-V 32 ビット基本整数命令セット**
  - **RV32IMAC: 整数乗除算、アトミック命令、および圧縮命令の標準拡張を備えた RISC-V 32 ビット基本整数命令セット**
  - **RV32IMAFC: 整数乗算/除算、アトミック命令、単精度浮動小数点、圧縮の標準拡張を備えた RISC-V 32 ビット基本整数命令セット**
- **[Select Processor Implementation]: デフォルトは 32-bit で、これはほとんどの組み込み RISC-V アプリケーションの標準です (2)。もう 1 つの選択肢は 64-bit です。**
- **[Implementation Optimization]: 内部パイプライン段数と、面積とパフォーマンス間の最適化トレードオフを設定します。デフォルトでは、[Microcontroller Preset] コンフィギュレーション (3) に対して [AREA] 最適化が選択されています。選択肢は次のとおりです。**
  - **AREA: 3 段パイプライン実装 — ロジック使用量を最小限に抑えます。**
  - **パフォーマンス: 5 ステージ パイプライン — パフォーマンスとリソース使用率のバランスを両立します。**

© Copyright 2026 Advanced Micro Devices, Inc.

- **FREQUENCY: 8 段パイプライン実装 — 追加のリソースを消費して、より高いクロック周波数の達成を目標とします。**

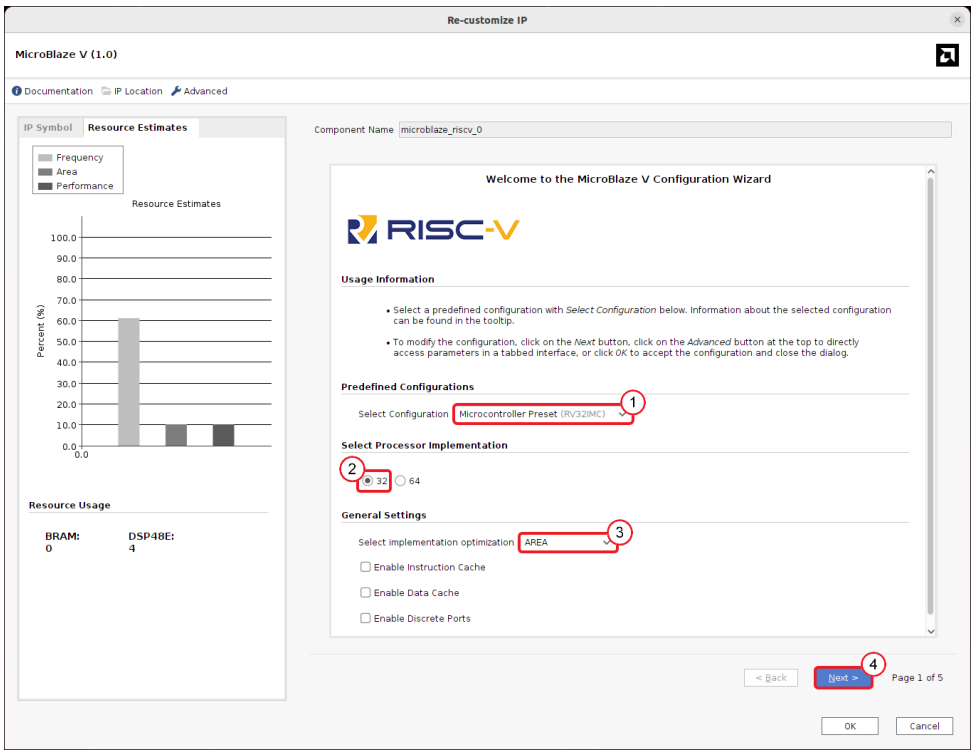


図 1-9: Welcome Page 設定の確認

2-1-3. デフォルト設定のまま **[Next]** をクリックし、**[General]** ページを開きます **(4)**。

2-1-4. **[General]** ページを確認し、デフォルト設定をメモします。

[MicroBlaze V Configuration Wizard] の [General] ページでは、命令セット、実行動作、オプションの信頼性機能を定義する主要なアーキテクチャ オプションにアクセスできます。これらの設定の多くは、事前に選択したプロセッサ プリセットから自動的に導出されます。

[Instruction and ISA-related options] (1) では、整数乗除算 (M)、アトミック命令 (A)、浮動小数点サポート (F)、コード圧縮 (C) などの RISC-V 拡張を有効または無効に設定できます。デフォルトでは、パフォーマンスの向上とコード サイズの削減のために乗除算およびコード圧縮が有効になっており、リソース使用量を最小限に抑えるために浮動小数点サポートは無効になっています。この構成は、マイクロコントローラー クラスの設計に適した RV32 ベースのプロファイルに対応します。

[Privilege mode and addressing options] (2) は、32 ビット アドレス空間の [Machine] モードに設定されています。このコンフィギュレーションにより、余分な

© Copyright 2026 Advanced Micro Devices, Inc.
AMD together we advance_

複雑さを回避しながら、ベアメタルおよび軽量な組み込みソフトウェアがサポートされます。

[Optimization options] (3) は実行効率を制御します。[Branch Target Cache] 設定を含む分岐最適化機能の設定が可能ですが、キャッシュ サイズは選択したプリセットで定義されたままになります。

[Fault-tolerance options] (4) は、セーフティ クリティカルなデザイン向けの保護メカニズムを提供しますが、コンフィギュレーションを軽量に保つため、デフォルトでは無効になっています。

利用可能な設定は相互に依存しており、選択したプロセッサ プリセットによって異なる場合があります。

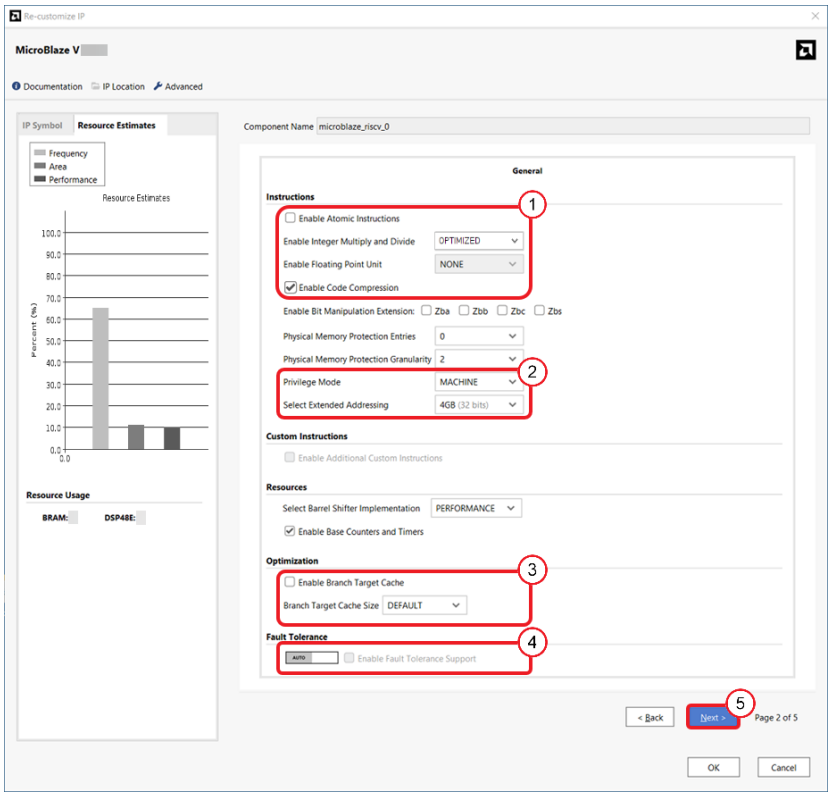


図 1-10: General ページ設定の確認

2-1-5. デフォルト設定のまま **[Next]** をクリックし、**[Exception]** ページ **(5)** を開きます。

[MicroBlaze V Configuration Wizard] の [Exceptions] ページでは、プログラム実行中にプロセッサが例外やトラップ状態を検出して応答する方法を設定します。これらのメカニズムにより、プロセッサはエラー状態を安全に処理し、ソフトウェア定義の例外ハンドラーに制御を移すことができます。

© Copyright 2026 Advanced Micro Devices, Inc.

このページのオプションでは、不正命令、アライメント違反のメモリー アクセス、演算フォルトなどのアーキテクチャ例外や、無効またはサポートされていないメモリー トランザクション中に発生するバス関連の例外のサポートを有効にします。

例外処理はシステムの堅牢性を向上させ、ソフトウェアの開発およびデバッグにおいて貴重な可視性を提供します。

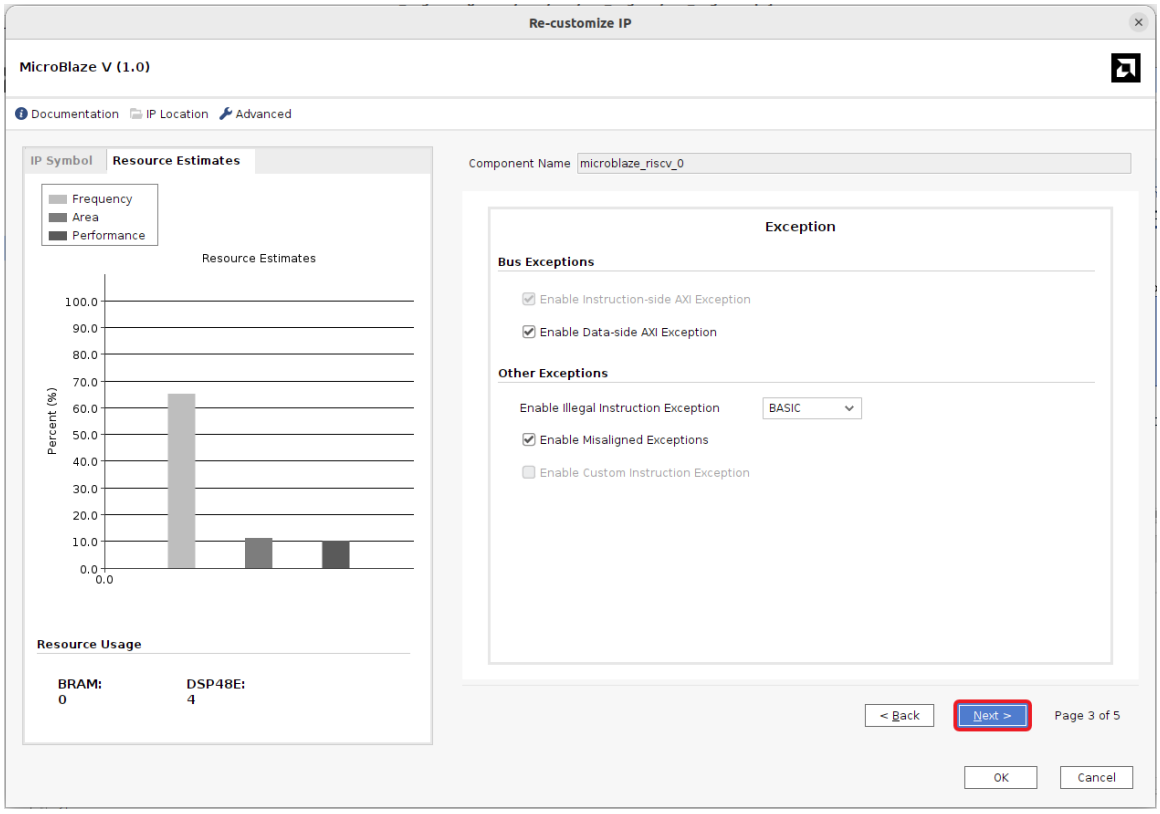


図 1-11: Exception Page 設定の確認

© Copyright 2026 Advanced Micro Devices, Inc.

2-1-6. デフォルト設定のまま **[Next]** をクリックし、**[Debug]** ページに進みます。

[MicroBlaze V Configuration Wizard] の [Debug] ページでは、ソフトウェア デバッグ、パフォーマンス解析、およびプログラム トレースに使用されるハードウェア機能を制御します。これらのオプションは、プロセッサが XSDB などの開発ツールとどのようにやり取りするかを定義し、ソフトウェア開発やシステムの立ち上げ時に利用可能な可観測性のレベルに影響を与えます。

[Debug Module Interface] は、プロセッサへの JTAG ベースのデバッグ アクセスを提供する MicroBlaze Debug Module V (MDM V) インターフェイスを有効にします。追加のオプションでハードウェア ブレークポイントおよびウォッチポイントを設定することにより、プログラム実行の正確な制御および確認が可能になります。

デバッグ リソースの数はデバッグのニーズに応じて調整できますが、デバッグ機能を強化すると達成可能な最大クロック周波数に影響する可能性があることに留意してください。

[Debug] ページには、パフォーマンス モニタリング、トレース、およびプロファイリングのオプションも含まれています。開発者はこれらの機能を使用して、実行統計の収集、レイテンシの測定、ならびにオンチップ バッファーまたは外部トレース インターフェイスを介したプログラム フロー情報のキャプチャを行うことができます。こ

© Copyright 2026 Advanced Micro Devices, Inc.

のような機能は、特に複雑なアプリケーションやタイミング制約の厳しいアプリケーションにおけるパフォーマンス チューニングやソフトウェアの最適化に役立ちます。

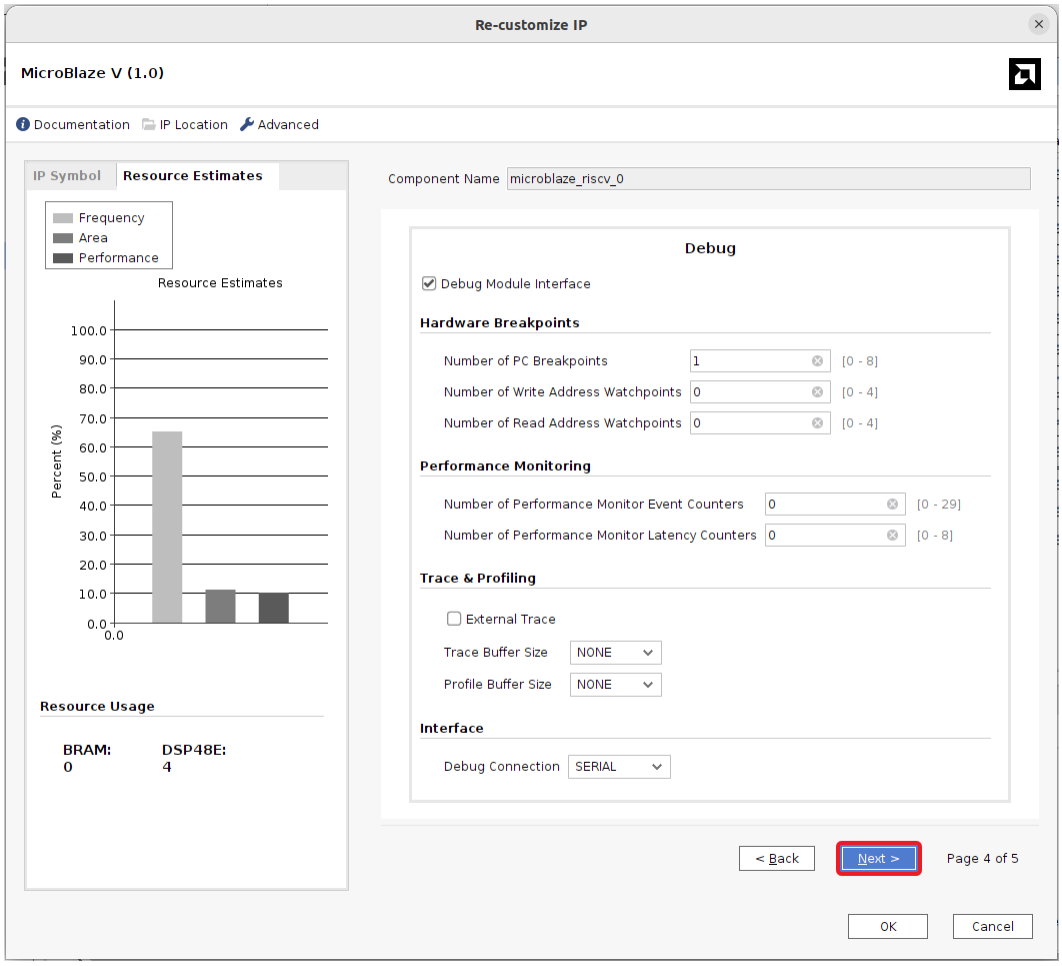


図 1-12: Debug Page 設定の確認

2-1-7. デフォルト設定のまま **[Next]** をクリックして **[Buses]** ページを開きます。

[MicroBlaze V Configuration Wizard] の [Buses] ページでは、プロセッサがメモリ、ペリフェラル、およびオプションのハードウェア アクセラレータにどのように接続するかを定義します。このページでは、データ転送、メモリ アクセス レイテンシ、およびシステム全体の統合を決定するローカルおよびシステム レベルのインターフェイスの有効/無効を制御します。

このページのコンフィギュレーション オプションには、オンチップ メモリへの低レイテンシ アクセスを可能にするローカル メモリ インターフェイスの有効化、外部メモリやペリフェラルと接続するための AXI ベース インターフェイスの選択、カスタム ロジックとの高スループット通信を実現するオプションのストリーミング インターフェイスのコンフィギュレーションが含まれます。追加のインターフェイス オプシ

© Copyright 2026 Advanced Micro Devices, Inc.
AMD together we advance_

ョンでは、デバッグ トレース、障害検出用のロックステップ動作、セーフティ クリティカルな設計で使用される監視インターフェイスなど、高度なユース ケースがサポートされます。

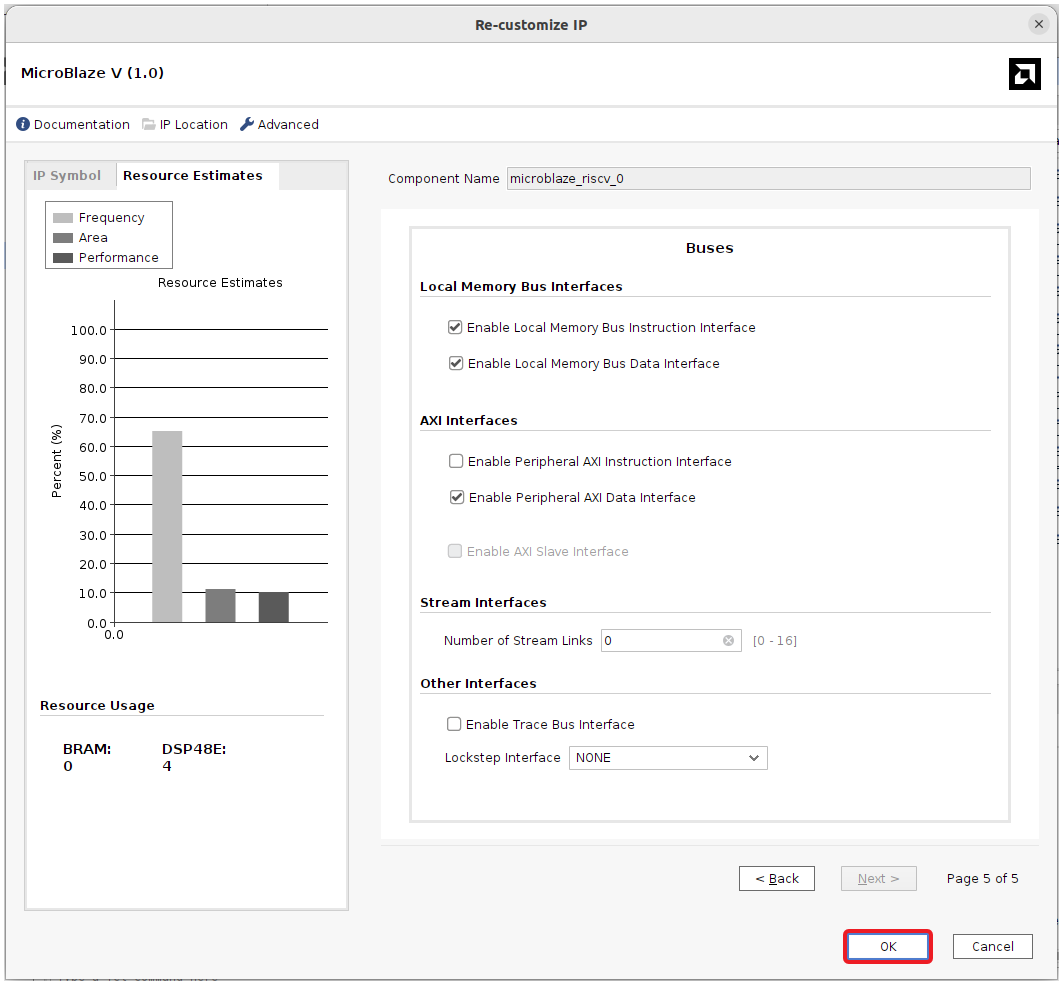


図 1-13: Buses ページの設定の確認

2-1-8. デフォルト設定のまま **[OK]** をクリックします。

© Copyright 2026 Advanced Micro Devices, Inc.

2-2. ブロック デザインを保存します。

2-2-1. **[File] > [Save Block Design As]** を選択します。

2-2-2. **[Save Block Design As]** ダイアログ ボックス **(1)** でデザインに **microblaze_v** と名前を付けます。

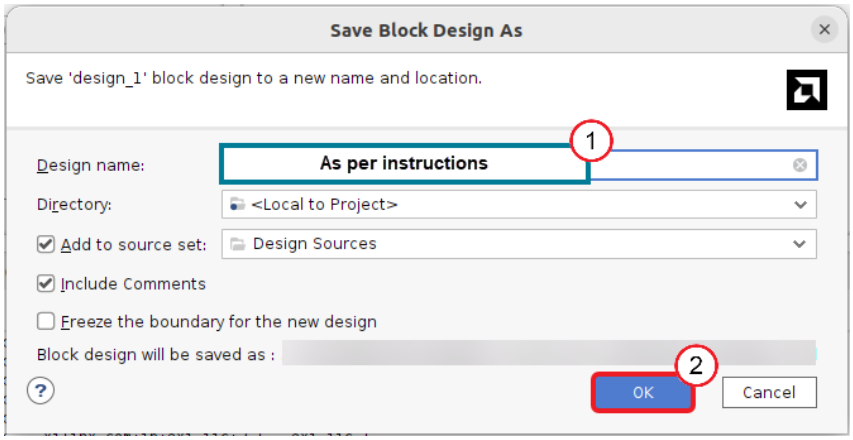


図 1-14: ブロック デザインの保存

2-2-3. **[OK]** をクリックしてブロック デザインを保存します **(2)**。

デバイス イメージの生成およびハードウェア プラットフォームのエクスポート 手順 3

3-1. 論理合成とインプリメンテーションを実行します。

3-1-1. **[Flow Navigator]** の **[Synthesis]** の下にある **[Run Synthesis]** をクリックします。

3-1-2. **[OK]** をクリックして **Run** を開始します。

© Copyright 2026 Advanced Micro Devices, Inc.
AMD together we advance_

または、[Flow] > [Run Synthesis] を選択するか、<F11> を押すこともできます。

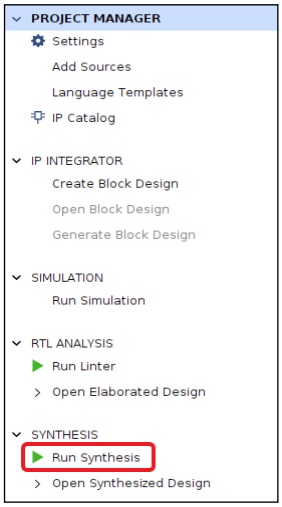


図 1-15: Run Synthesis の選択

論理合成の実行中は、右上のウィンドウにステータス バーが表示されます。このステータス バーは、デザイン プロセス全体を通してさまざまな処理に応じて回転表示されます。ステータス バーは、バックグラウンドでプロセスが実行中であることを示しています。

合成プロセスが完了すると、[Synthesis Completed] ダイアログ ボックスが開きます。このダイアログ ボックスで、インプリメンテーションを実行するか、合成済みデザインを開くか、レポートを表示するかを選択します。

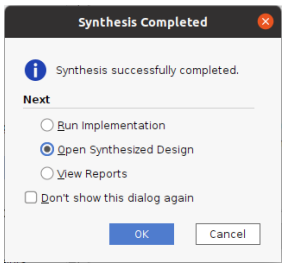


図 1-16: Synthesis Completed ダイアログ ボックス

以下に示すインプリメンテーション、デバイス イメージ (PDI) 生成、およびハードウェアのエクスポートの手順は、マシンの構成によって異なりますが、完了までに約 5 ～ 7 分かかる場合があります。時間の制約によりこれらの手順を実行できない場合は、「Creating a Bare-Metal Application Using the Vitis Unified IDE」の手順に直接進むことができます。

© Copyright 2026 Advanced Micro Devices, Inc.

[Vivado IDE] からエクスポートされたハードウェア XSA ファイルは support ディレクトリに用意されている点に注意してください。このファイルは、[Vitis Unified IDE] でのプラットフォーム作成に必要です。

3-1-3. **[Synthesis Completed]** ダイアログ ボックスで **[Run Implementation]** をクリックします。

3-1-4. **[OK]** を **2** 回クリックしてインプリメンテーションを開始します。

3-1-5. インプリメンテーションが正常に完了したら、**[Implementation Completed]** ダイアログ ボックスで **[Cancel]** をクリックします。

インプリメント済みデザインを開くと、Vivado IDE はデザイン解析のためにネットリスト、デザイン制約、ターゲット パーツ、および配置配線の結果を Vivado IDE にインポートします。[Flow Navigator] から、[Timing Summary Report] や [Utilization Report] などの最も重要なレポートを参照できます。

3-2. デバイス イメージを生成します。

3-2-1. **[Flow Navigator]** の **[Program and Debug]** の下にある **[Generate Device Image]** をクリックします。

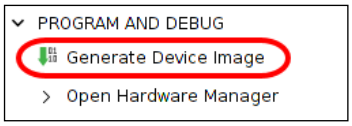


図 1-17: デバイス イメージの生成

3-2-2. **[OK]** をクリックします。

生成が完了すると、[Device Image Generation Completed] ダイアログ ボックスが開きます。

3-2-3. **[Cancel]** をクリックしてウィンドウを閉じます。

デバイス イメージを生成したら、次の手順としてハードウェアを XSA ファイルとしてエクスポートします。このファイルは、Vitis Unified IDE ツールの起点として使用されます。

なお、生成済みの XSA ファイルは、次のディレクトリに用意されています。

`$TRAINING_PATH/microblaze_v/support`

3-3. ハードウェアをエクスポートします。

3-3-1. **[File] > [Export] > [Export Hardware]** を選択します。

© Copyright 2026 Advanced Micro Devices, Inc.
AMD together we advance_

3-3-2. **[Export Hardware Platform]** ダイアログ ボックスで **[Next]** をクリックします。

3-3-3. **[Output]** ダイアログ ボックスで **[Include device image]** を選択します **(1)**。

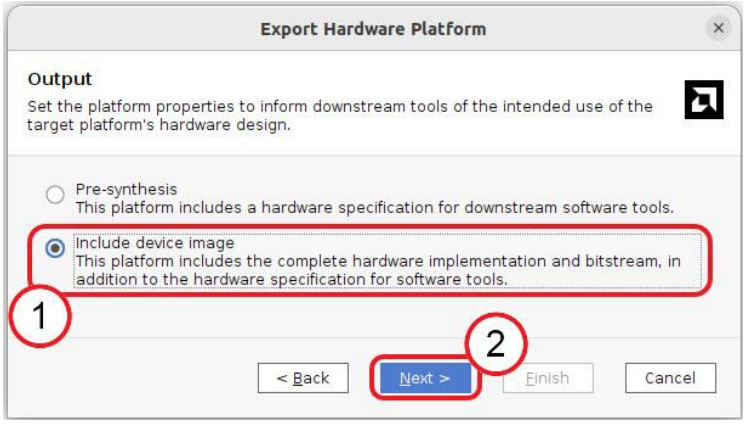


図 1-18: ハードウェア プラットフォームのエクスポートとデバイス イメージの取り込み

3-3-4. **[Next] (2)** をクリックします。

3-3-5. **XSA** ファイル名として **microblaze_v_preset_wrapper** を入力します **(1)**。

3-3-6. **[Export to]** フィールドに、**XSA** ファイルを書き出すパスを入力します **(2):**

`$TRAINING_PATH/microblaze_v/lab/hw`

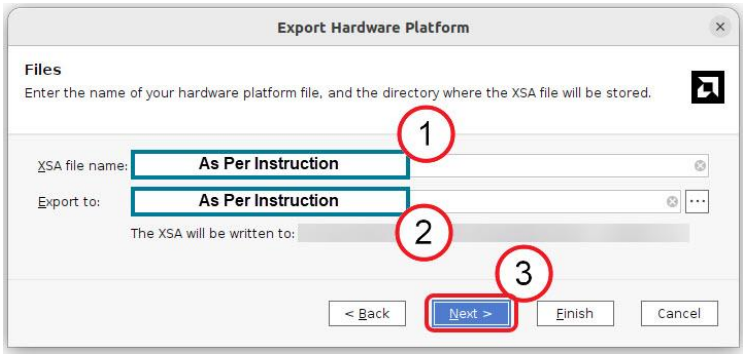


図 1-19: XSA の名前とパスの選択

3-3-7. **[Next] (3)** をクリックします。

3-3-8. **[Finish]** をクリックして **XSA** ファイルを生成します。

[Tcl Console] で “Successfully created hardware platform” メッセージを確認します。

3-4. プロジェクトを閉じます。

3-4-1. **[File] > [Project] > [Close]** を選択してプロジェクトを閉じます。

© Copyright 2026 Advanced Micro Devices, Inc.

[Close Project] ダイアログ ボックスが開きます。

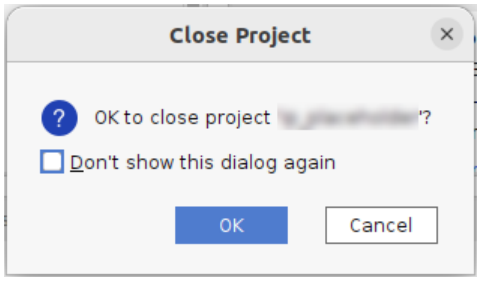


図 1-20: [Close Project] ダイアログ ボックス

3-4-2. **[OK]** をクリックします。

3-5. Vivado Design Suite を終了します。

3-5-1. **[File] > [Exit]** を選択します。

[Exit Vivado] ダイアログ ボックスが開きます。



図 1-21: Exit Vivado ダイアログ ボックス

3-5-2. プロジェクトまたはプロジェクトの一部の保存を求められた場合は、保存するプロジェクトの要素を選択し、**[Save]** をクリックして選択した要素を保存します。保存しない場合は、**[Don't Save]** をクリックします。

3-5-3. **Vivado Design Suite** を終了するか確認されたら、**[OK]** をクリックします。

注: Vivado Design Suite の終了時に確認メッセージが表示されないようにするには、[Don't show this dialog again] オプションを選択できます。

© Copyright 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

Vitis Unified IDE を使用したベアメタル アプリケーションの作成 手順 4

この手順では、まず [Vitis Unified IDE] を開いてディスク上でプロジェクトが保存されるワークスペースを指定します。その後、プラットフォーム コンポーネントを作成し、アプリケーション コンポーネントを作成して、アプリケーションをビルドします。

4-1. Vitis Unified IDE 環境をセットアップします。

4-1-1. **<Ctrl + Alt + T>** を押して、新しいターミナル ウィンドウを開きます。

4-1-2. 次のコマンドを入力して、**Vitis** ツールの環境を設定します:

```
[host]$ source /opt/amd/2025.2/Vitis/settings64.sh
```

注記: カスタマー トレーニング環境 (CustEd_VM) では、Vitis ツールのインストール パスが /opt/amd/2025.2/Vitis に設定されています。ご使用の環境でツールが別の場所にインストールされている場合は、そのインストール パスを使用してください。

4-2. Vitis Unified IDE を起動します。

4-2-1. 次のコマンドを入力して **Vitis Unified IDE** を起動します:

```
[host] $ vitis
```

4-2-2. **[Set Workspace]** をクリックして、ワークスペース ディレクトリを設定します。

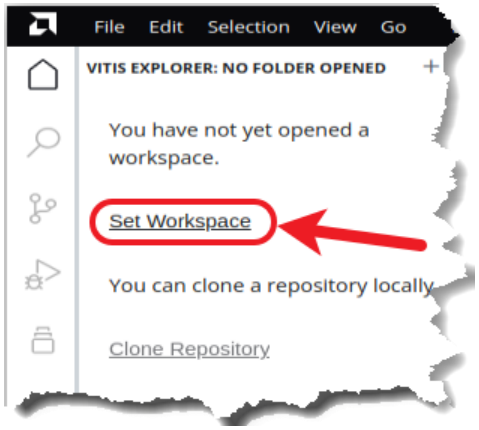


図 1-22: Vitis Unified IDE での Workspace の設定

4-2-3. 次のディレクトリに移動します:

```
$TRAINING_PATH/microblaze_v/lab/sw
```


© Copyright 2026 Advanced Micro Devices, Inc.

注: [Vitis IDE] GUI は環境変数をサポートしていないため、$TRAINING_PATH をフル パスに置き換える必要があります。この場合、$TRAINING_PATH は /home/amd/training ディレクトリに置き換えてください。

4-2-4. **[Open]** をクリックします。

プラットフォーム コンポーネントにはハードウェア設計の詳細な記述が含まれており、次のものが含まれます:

- 利用可能なプロセッサ
- PL ベースのペリフェラルおよび有効化された PS ペリフェラル
- システム全体のメモリ マップ

この記述に基づいて、ボード サポート パッケージ (BSP) などのソフトウェアやアプリケーションをハードウェアに合わせてカスタマイズできます。以下の手順では、カスタム ハードウェアを含める方法を説明します。

4-3. プラットフォーム コンポーネントを作成します。

4-3-1. ツールバーに移動し、**[File] (1)** をクリックします。

4-3-2. **[New Component]** にカーソルを合わせ、オプションから **[Platform]** を選択します **(2)**。

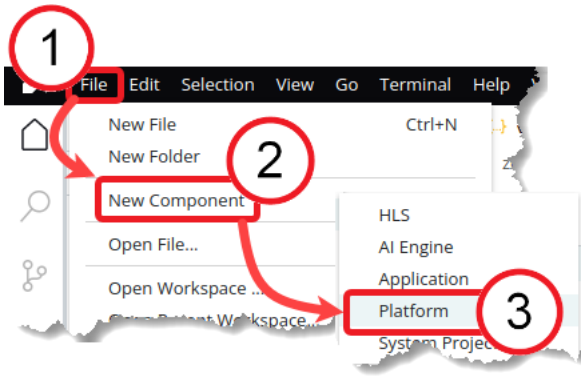


図 1-23: Create Platform Component ウィザードを開く

[Create Platform Component] ダイアログ ボックスが表示されます。

注: このウィザードにアクセスする方法は他にもありますが、これが最も簡単です。

© Copyright 2026 Advanced Micro Devices, Inc.
AMD together we advance_

4-3-3. **[Component name]** フィールド **(1)** に **hello_world_plat** を入力します。

ヒント: このフィールドにスペースが含まれていないことを確認してください。

これによりプラットフォーム コンポーネントの名前が設定され、現在のワークスペース内に関連するディレクトリとファイルが生成されます。

[Component location] フィールドには、自動的にワークスペースの場所が設定されることに注意してください。別の場所にコンポーネントを作成したい場合は、この場所を変更できます。

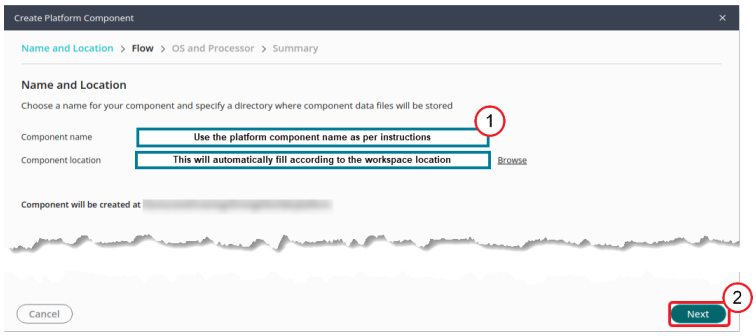


図 1-24: プラットフォーム コンポーネント名とロケーションの入力

4-3-4. **[Next]** をクリックして、ハードウェア記述ファイルの場所の指定 **(2)** に進みます。

4-3-5. **[Hardware Design]** オプション **(1)** を選択します。

4-3-6. **[Hardware Design (XSA)]** フィールド **(2)** の横にある **[Browse]** をクリックします。

4-3-7. 次のパスを参照します:

[自身で生成した XSA を使用する場合]:

```
$TRAINING_PATH/microblaze_v/lab/hw
```

[ビルド済み XSA の使用]:

```
$TRAINING_PATH/microblaze_v/support
```

4-3-8. **microblaze_v_preset_wrapper.xsa** をダブルクリックします。

© Copyright 2026 Advanced Micro Devices, Inc.

[Vitis Unified IDE] では環境変数が自動的に展開されないことに注意してください。パスを直接入力するか、そのパスまで移動して指定する必要があります。環境変数の詳細については、この演習の Introduction セクションを参照してください。

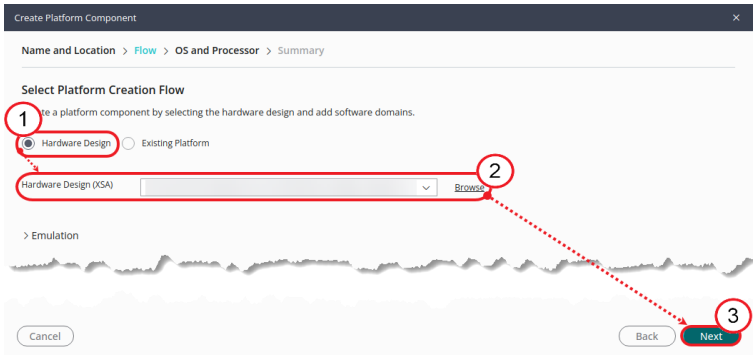


図 1-25: プラットフォーム作成フローの選択

4-3-9. **[Next] (3)** をクリックします。

4-3-10. まだ選択されていない場合は、**[Operating system]** ドロップダウン リストから **[standalone]** を選択します **(1)**。

注: オプションが自動的に入力されるまで、しばらく待つ必要がある場合があります。

4-3-11. 自動的に入力されていない場合は、**[Processor]** ドロップダウン リストから **microblaze_riscv_0** を選択します **(2)**。

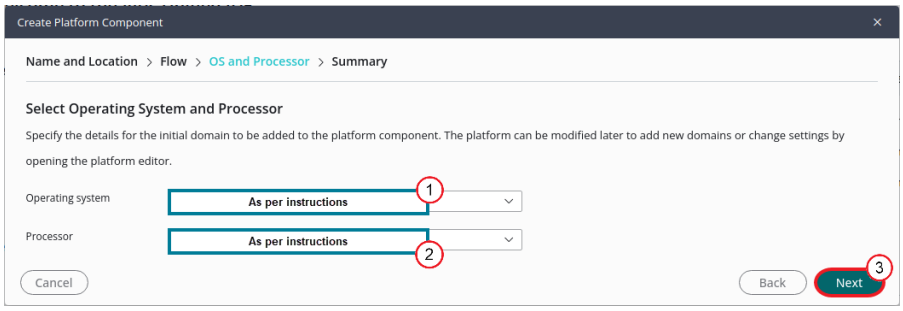


図 1-26: オペレーティング システムとプロセッサの選択

4-3-12. **[Next]** をクリックして次の手順 **(3)** に進みます。

4-3-13. プラットフォーム コンポーネントの情報を確認し、**[Finish]** をクリックします。

注記: 処理には数分かかります。通常、ステータスは左下隅に表示され、進捗メッセージは [Output] ウィンドウに表示されます。

© Copyright 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

プラットフォーム コンポーネントとともに、ウィザードは選択されたオペレーティング システムとプロセッサに基づいて、プラットフォーム コンポーネント内にドメインを作成します。

プラットフォーム コンポーネントを作成したら、次の手順はアプリケーション コンポーネントの作成です。

[Create Application Component] ウィザードを使用すると、Vitis Unified IDE で利用可能なサンプル テンプレートを使用して、アプリケーションをすばやく作成できます。選択したテンプレートに基づいて適切なオプションが表示されるため、適切なオプションを選択してアプリケーション コンポーネントを作成します。

4-4. アプリケーション コンポーネントを作成します。

4-4-1. ツールバーに移動し、**[File] (1)** をクリックします。

4-4-2. オプション **(2)** から **[New Example]** を選択します。

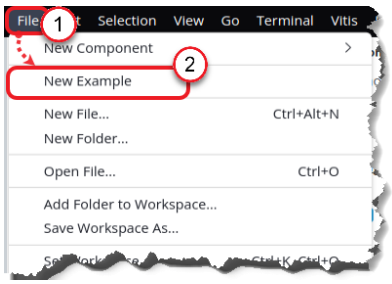


図 1-27: Examples からの Application Component の作成

[Examples] ウィンドウが表示されます。

この演習では、テンプレートから構築済みのサンプルの 1 つを使用します。Vitis Unified IDE でアプリケーション コンポーネントを作成し、それをハードウェア ボード上で実行して出力を確認する方法について学びます。

4-4-3. **[Examples]** ウィンドウから **Hello World** サンプルを検索します **(1)**。

4-4-4. **[Embedded Software Examples]** の下にある **[Hello World]** を選択します **(2)**。

© Copyright 2026 Advanced Micro Devices, Inc.

4-4-5. **[Create Application from Template]** をクリックして、アプリケーション コンポーネント **(3)** を作成します。

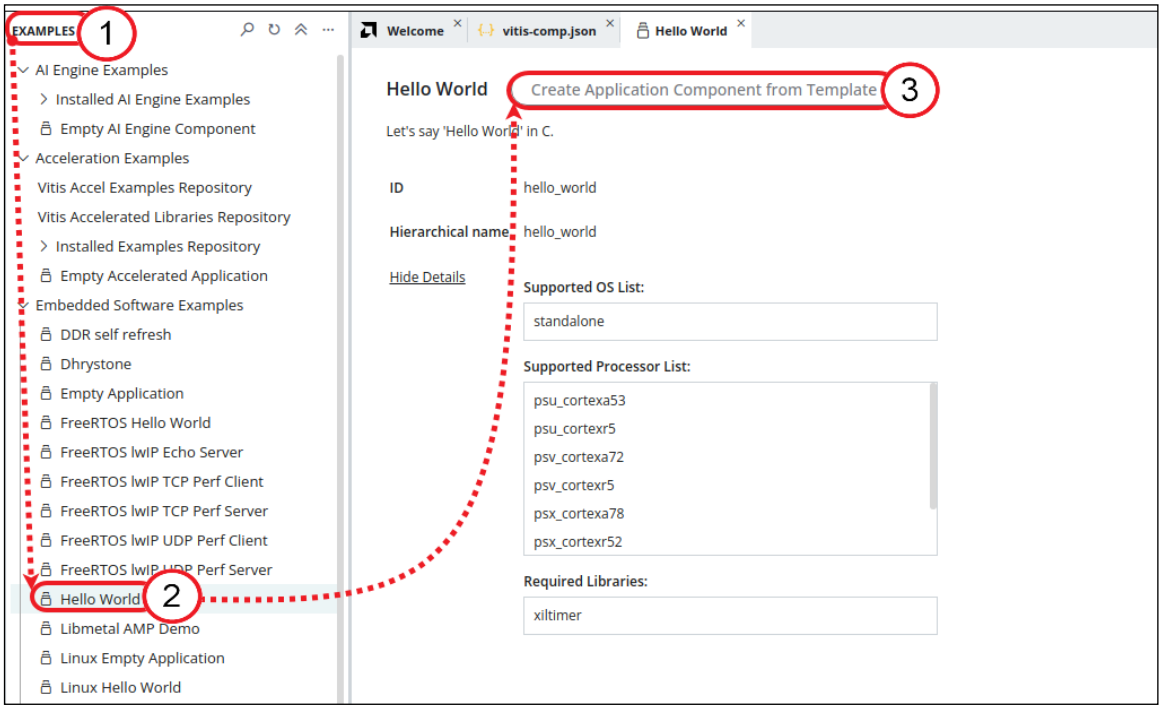


図 1-28: サンプル テンプレートの選択とアプリケーション コンポーネントの作成

[Create Application Component - Hello World] ダイアログ ボックスが表示されます。

© Copyright 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

4-4-6. **[Component name]** フィールド **(1)** に **hello_world_app** と入力します。

4-4-7. **[Component location]** フィールドの横にある **[Browse]** をクリックし、適切な場所を設定します **(2)**。

4-4-8. 次のディレクトリを参照します**:**

```
$TRAINING_PATH/microblaze_v/lab/sw
```

4-4-9. **[OK]** をクリックします。

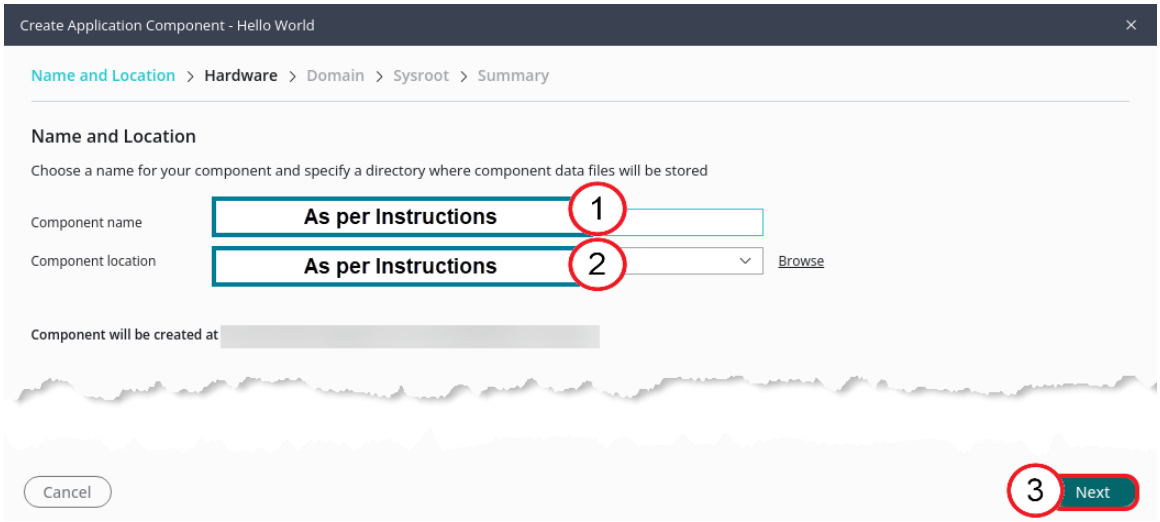


図 1-29: Application Component Name and Location の指定

4-4-10. **[Next]** をクリックして、デバイス**/**プラットフォームを選択します **(3)**。

4-4-11. リストから **[hello_world_plat]** を選択します。

4-4-12. **[Next]** をクリックして、プラットフォームで利用可能なドメインからドメインを選択します。

4-4-13. デフォルト ドメインの詳細を確認します**:**

- [Domain name]: standalone_microblaze_riscv_0
- OS: [standalone]
- [Processor]: microblaze_riscv_0

4-4-14. **[Next]** をクリックしてデフォルトのドメイン設定を受け入れ、**[Summary]** ページに進みます。

4-4-15. **[Summary]** を確認し、**[Finish]** をクリックします。

© Copyright 2026 Advanced Micro Devices, Inc.

4-5. アプリケーション コンポーネントをビルドします。

4-5-1. **[Flow]** ウィンドウに移動します。

4-5-2. **[Build]** オプションをクリックして、**hello_world_app** アプリケーション コンポーネントをビルドします。

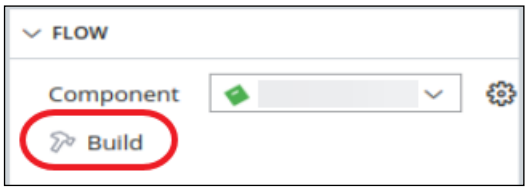


図 1-30: コンポーネントのビルド

注: [Application - Platform Build Dependency] ポップアップ ウィンドウが表示された場合は、[Always build platform with application] オプションを選択し、[Save in User Preference] をクリックします。

4-5-3. **[Output]** ウィンドウに **"Build Finished successfully"** メッセージが表示されることを確認します。

ボード上でのアプリケーションの実行 手順 5

Vitis 環境では、Spartan UltraScale+ 評価ボード上でアプリケーションを実行するために必要な前提ファイルが自動的に生成されます。実機ハードウェアを用意できる場合は、以下の手順に進んでアプリケーションを実行できます。用意できない場合は、このセクションを参照して JTAG 経由でデザインを実行するプロセスを確認してください。

On-Demand/CloudShare ユーザーの方で、評価ボードがローカルで利用可能な場合は、必要なプロジェクト ファイルをローカル パスにコピーし、以下の手順を進めてください。また、On-Demand Portal の [FAQ] セクションにある「Can I run the labs on my local machine?」の質問も参照してください。

それ以外の場合は、これらの手順を実行せずに確認するだけで構いません。

これで、JTAG ブート モードを使用してターゲットの Spartan UltraScale+ ボードをプログラムし、アプリケーションをダウンロードして、プログラムが正常に動作するかテストします。

© Copyright 2026 Advanced Micro Devices, Inc.
AMD together we advance_

5-1. SCU35 ボードを起動します。

以下の SCU35 評価ボードの図は、代表的な機能の一部を示しています。基板 (PCB) 上におけるこれらの機能の位置を確認してください。ボードを適切に動作させるため、使用前にケーブルの接続やスイッチの設定 (または確認) を行う必要があります。

以降の手順では、番号が付けられた PCB 上の位置にある項目を参照します。詳細は、SCU35 Evaluation Board User Guide (UG1713) を参照してください。

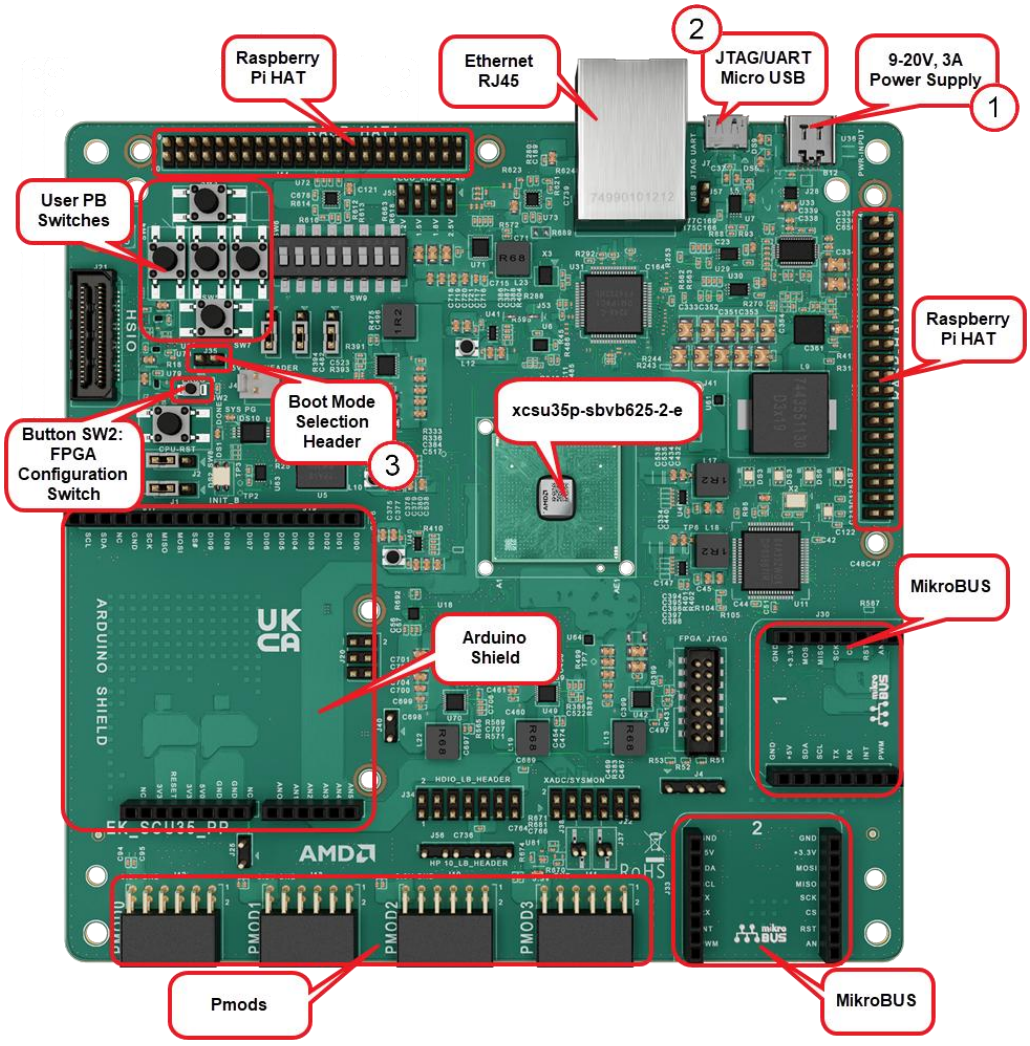


図 1-31: SCU35 の概要

5-1-1. 電源コネクタが接続されていないことを確認します **(1)**。

5-1-2. ボードとホストの間に **micro-USB JTAG/UART** ケーブルを接続します **(2)**

© Copyright 2026 Advanced Micro Devices, Inc.

5-1-3. ボード上の **J35** の位置を確認します **(3)**。

このヘッダーは Spartan UltraScale+ デバイスのブート モードを設定するもので、JTAG に設定する必要があります。

5-1-4. ボードが **JTAG** からブートするように設定されていることを確認するため、以下に示すように **J35** を設定します。

ブート モード	モード ピン [3:0]	モード J35
JTAG	101 / 0x5	[Open]
QSPI SPI_24	100 / 0x4	ジャンプ

5-1-5. 電源コネクタ **(1)** を接続します。

これで、ボードをコンフィギュレーションする準備が整いました。

5-2. 開発ボード上の USB シリアル ポートに接続されている COM ポートを確認します。

5-2-1. 次のように **USB** の適切なチャネルを特定します。

Customer Education VM を使用している場合は、[VirtualBox Manager] で [Devices] > [USB] > [Xilinx SCU35] を選択します。

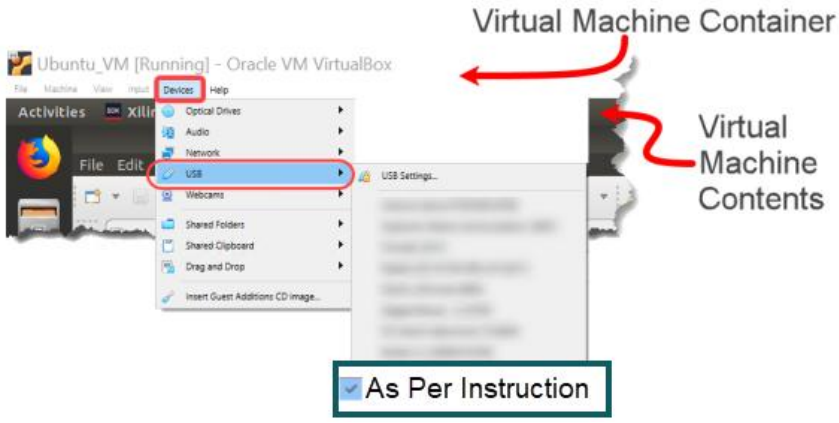


図 1-32: ハードウェア ターゲットの選択

これにより、VM が USB 接続に確実にアクセスできるようになり、適切な USB ポートがホスト/VM 間の境界を越えることが許可されます。

© Copyright 2026 Advanced Micro Devices, Inc.
AMD together we advance_

5-3. SCU35 ボードをプログラムします。

5-3-1. **[Vitis Unified IDE]** に戻り、ツールバーの **[Vitis]** にカーソルを合わせます **(1)**。

5-3-2. オプション **(2)** から **[Program Device]** を選択します。

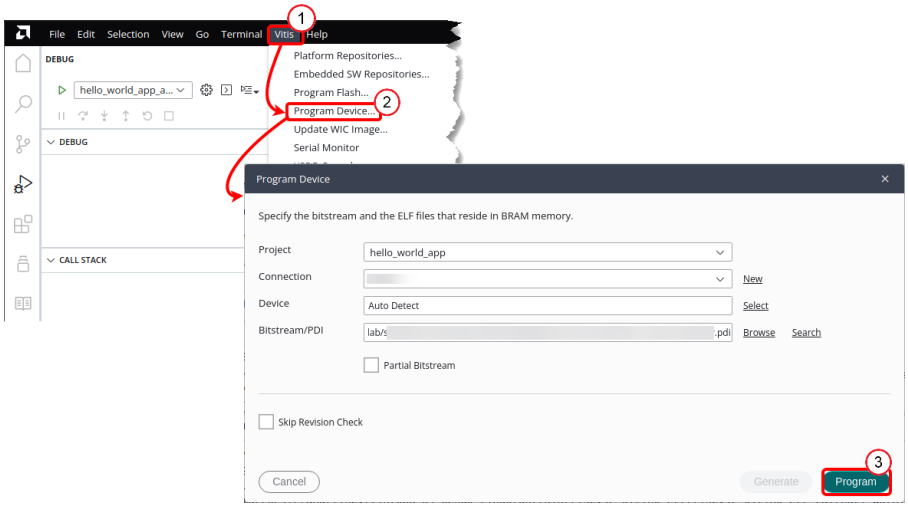


図 1-33: SCU35 のプログラミング

[Program Device] ダイアログ ボックスが表示されます。

5-3-3. **[Program]** をクリックしてデフォルト設定で続行し、デバイスのプログラミングを開始します **(3)**。

[GTK Term] は、シリアル ポート/UART 通信のみをサポートするインターフェイスです。より汎用的な [Terminal] タブは、SSH や Telnet などの他の形式もサポートしています。

シリアル ポート通信はエニュメレーションされるインターフェイスである USB 経由で行われるため、ポートを認識させるには、ケーブルを接続した状態でボードの電源を投入する必要があります。

VM を使用している場合は、Zynq UltraScale+ MPSoC/Versal アダプティブ SoC ボード向けに [VirtualBox Manager] で [Devices] > [USB] > [Xilinx <board selection> [0800]] を選択するか、または [Devices] > [USB] > [Digilent Adept USB Device [0700]] を選択して、適切な USB ポートがホスト/VM 境界を越えて接続されることを確認してください。

© Copyright 2026 Advanced Micro Devices, Inc.

5-4. GTKTerm を起動し、ポート設定を行います。

5-4-1. **[quick launch]** ツールバーから **[GTKTerm]** アイコン **()** をクリックします。

あるいは、Linux ターミナル ウィンドウ (<Ctrl + Alt + T>) から次のように入力して GTKTerm を起動することもできます:

```
[host] $ sudo gtkterm
```

注記: アプリケーションは通常のユーザーとして実行できますが、ポートにアクセスするにはスーパーユーザーである必要があります。

[GTKTerm] ウィンドウが開いたら、次の操作を実行します。

5-4-2. **[Configuration] > [Port]** を選択して、**[Configuration]** ダイアログ ボックスを開きます。

5-4-3. ボードに関連付けられているポートを特定し、ポートを **/dev/ttyUSBx (x** は **0**、**1**、**2**、**3** など**)** に設定します。

5-4-4. ボー レートを **115200** に設定します。

5-4-5. その他の設定はデフォルトのままにします。

注: UART がどのポートに接続されているか特定できない場合は、/dev/USBx のインスタンスを複数開くことができます。

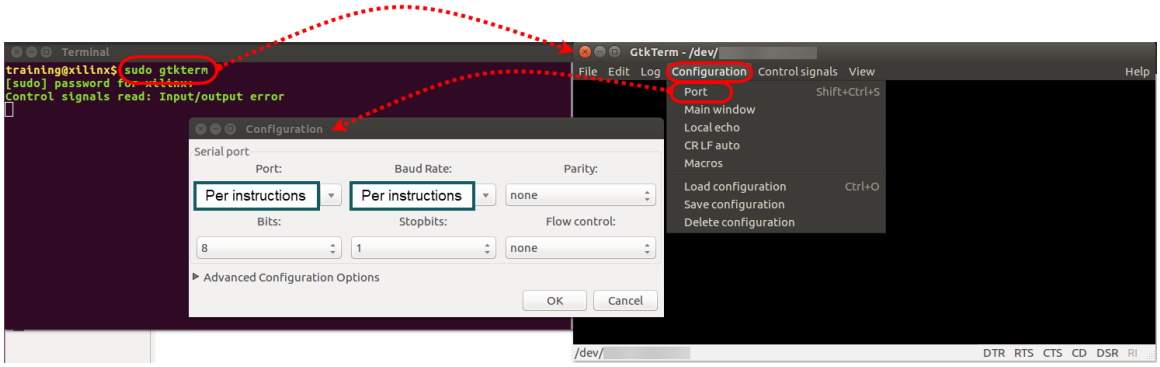


図 1-34: GtkTerm の起動と Port Configuration の選択

[オプション]: [Configuration] > [Save configuration] を選択してこれらの設定を保存すると、GKTTerm を開くたびに再設定する必要がなくなります。

コンフィギュレーションを "default" として保存すると、GTKTerm の起動時にこのコンフィギュレーションが開きます。あるいは、別の名前でコンフィギュレーションを保存することもできますが、その場合は GTKTerm を起動するたびにコンフィギュレーションをロードする必要があります。

5-4-6. **[OK]** をクリックして設定を保存し、ターミナルを開いたままにしておきます。

© Copyright 2026 Advanced Micro Devices, Inc.
AMD together we advance_

5-5. [auto CR/LF mode] を有効にします。

GTKTerm は、CR (carriage return) 文字から LF (line feed) 文字への自動変換、およびその逆の変換をサポートするターミナル エミュレーターです。

5-5-1. **[Configuration] > [CR LF auto]** を選択して、自動 **CR/LF** モードを有効にします。

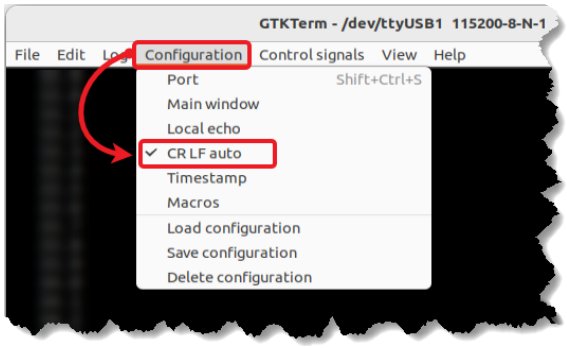


図 1-35: GTKTerm での CR/LF Auto の有効化

注: この手順はオプションですが、ターミナル上でシリアル メッセージが整列して表示されるようになります。

5-6. ハードウェア上でアプリケーションを実行します。

5-6-1. ツールバー メニューから **[Debug]** アイコンをクリックします **(1)**。

5-6-2. **[Start Debugging]** アイコンをクリックして、デバッグ セッションを開始します **(2)**。

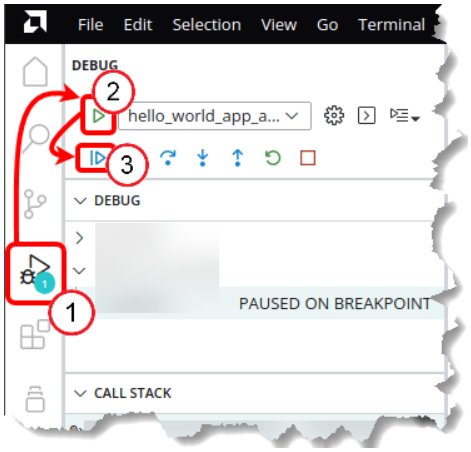


図 1-36: デバッグの開始と実行

これにより、ハードウェアがプログラムされ、デバッグ セッションが開始されます。

5-6-3. ツールバーの **[Continue]** アイコンをクリックして、プログラムを実行します **(3)**。

© Copyright 2026 Advanced Micro Devices, Inc.

5-6-4. **[GTKTerm]** シリアル ターミナルを選択して、アプリケーションからの出力を確認します。

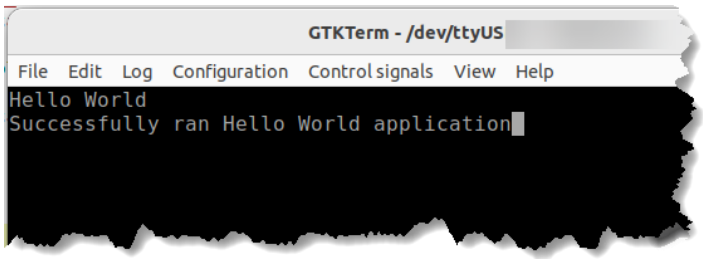


図 1-37: アプリケーションの出力

5-6-5. **[GTKTerm]** アプリケーションを終了します。

5-6-6. 評価ボードの電源をオフにします。

5-7. [Vitis Unified IDE] を閉じます。

5-7-1. **[File] > [Close Window]** を選択してツールを閉じます。

5-8. [Terminal] ウィンドウを閉じます。

5-8-1. **[Terminal]** ウィンドウに戻ります。

5-8-2. **exit** と入力して **[Terminal]** ウィンドウを閉じます。

一部のシステム (特に VM) では、メモリが制限されている場合があります。ワークスペースを削除するとディスク領域の一部が解放され、他の演習を実行できるようになります。

実行した演習が含まれるディレクトリは、グラフィカル インターフェイスまたはコマンドライン インターフェイスを使用して削除できます。どちらの方法を選択してもかまいません。どちらの処理でも、$TRAINING_PATH/microblaze_v ディレクトリ内のすべてのファイルが再帰的に削除されます。

5-9. [オプション] [ローカル VM のみ—CloudShare は対象外] ファイル システムをクリーンアップします。

GUI を使用する場合:

5-9-1. **$TRAINING_PATH/microblaze_v** へ移動します。

5-9-2. **[microblaze_v]** を選択します。

© Copyright 2026 Advanced Micro Devices, Inc.

5-9-3. **[Delete]** を押します。

-- または --

[Linux ユーザー]: コマンド ラインを使用:

5-9-4. **<Ctrl + Alt + T>** を押してターミナル ウィンドウを開きます。

5-9-5. 次のコマンドを入力して、ワークスペースの内容を削除します:

```
[host] $ rm -rf $TRAINING_PATH/microblaze_v
```

まとめ

この演習では、Vivado のサンプル プロジェクト機能を使用して MicroBlaze V ソフト プロセッサ システムを作成し、Spartan UltraScale+ FPGA SCU35 Evaluation Kit をターゲットにする方法を学びました。プリセット、ISA 拡張、例外処理、デバッグ機能、バス インターフェイスなど、MicroBlaze V プロセッサで利用可能な包括的なコンフィギュレーション オプションを確認しました。その後、デバイス イメージを生成し、ハードウェア プラットフォームを XSA ファイルとしてエクスポートしました。

さらに、[Vitis Unified IDE] で "Hello World" ベアメタル アプリケーションを作成し、MicroBlaze V プロセッサ向けにビルドして、SCU35 ボード上で実行する方法を学びました。

© Copyright 2026 Advanced Micro Devices, Inc.